Deployment

Gordon Oboh

Department of Data Science, Monroe College, King Graduate School

CS703-152W: Applied Data Science Project

Professor Nicholas Nardi

July 7, 2026

Deployment

6 Deployment

6.1 Plan Deployment

Although this academic project does not involve deploying a live production system, this section outlines a deployment plan that would apply if the Random Forest model were to be used in a real-world setting.

6.1.1 Model Packaging and Serialization

The trained Random Forest model would be serialised using `joblib`, preserving the fitted pipeline including the TargetEncoder transformations and the K-Means location cluster mapping. This serialised model file would serve as the deployable artefact, versioned alongside the project code in a git repository.

6.1.2 Deployment Architecture

A lightweight REST API would be the most practical deployment approach for this project. A Flask or FastAPI application would expose a single prediction endpoint accepting listing features as JSON input and returning a predicted monthly rent. The API would load the serialised model at startup and perform all necessary preprocessing (imputation of missing amenity fields, encoding of categorical variables, and location cluster assignment) before generating a prediction. The API would be containerised using Docker and hosted on a cloud platform such as AWS Elastic Beanstalk, Heroku, or Render. This approach keeps infrastructure costs near zero for low traffic while remaining scalable if usage grows. A simple web form frontend could be added as an optional layer, allowing non-technical users to enter property details (square footage, number of bedrooms, amenities, location) and receive an instant price estimate.

6.1.3 Documentation Requirements

Three pieces of documentation would be produced for deployment:

- **Technical Documentation:** Instructions for loading the model, running the API locally, and deploying to a cloud platform. This would include environment setup, dependency

management (Python packages listed in `requirements.txt`), and configuration options.

- **User Guide:** A brief guide for stakeholders explaining what the model predicts, what data it requires, how to interpret the output, and what its limitations are: specifically that predictions are less reliable for high-price listings.
- **Data Dictionary:** A reference listing all input features, their types, allowed values, and whether they are required or optional, so that users can prepare data correctly.

6.1.4 Training and Support

A short orientation session would be provided to any stakeholders who would use the system directly. This would cover how to submit a prediction request, how to read the results, and what to do if the output seems unreasonable. Ongoing support would be handled through a single point of contact (the project lead), with a documented escalation path for data quality or model performance issues.

6.2 Plan Monitoring and Maintenance

A monitoring and maintenance plan is essential to ensure the model remains accurate and useful over time, since housing market conditions, listing practices, and rental prices all change.

6.2.1 Performance Monitoring

Once deployed, the model's prediction accuracy should be tracked continuously. The key metrics to monitor are the same ones used during evaluation:

- **Mean Absolute Error (MAE):** The average dollar deviation between predicted and actual rent. A significant increase from the baseline of $35.58 would signal that the model has degraded.
- **R^2:** The proportion of variance explained. A drop from the baseline of 0.946 would indicate that the model is losing its predictive power.

- **Prediction drift:** The average prediction across all new requests should remain reasonably close to the training-period average of approximately $1,016. A sustained shift could indicate that the market has changed.

These metrics would be calculated on a rolling basis as actual rental prices become available. A dashboard (or a simple weekly report) would flag any metric that exceeds a predefined threshold: for example, a MAE increase of more than 20% above the baseline.

6.2.2 Data Drift Detection

Even before ground-truth prices are available, the model's input data should be monitored for drift. If the distribution of features such as square footage, number of bedrooms, or location shifts significantly from the training set, the model's predictions may become unreliable even if the model itself has not changed. Automated scripts would compare incoming data distributions to the training set on a monthly basis using statistical tests (e.g., Kolmogorov-Smirnov for numeric features, chi-square for categorical features). Any significant drift would trigger an alert and prompt a review of whether retraining is needed.

6.2.3 Retraining Schedule

The model should be retrained on a regular schedule to remain current with market conditions. A quarterly retraining cycle is recommended, with the following triggers for off-cycle retraining:

- A sustained MAE increase of more than 20% above baseline.
- A statistically significant data drift alert persisting for two consecutive months.
- A major market event (e.g., a significant shift in regional housing policy or economic conditions).

Retraining would follow the same pipeline used in the original project: data collection, cleaning with region-based IQR filtering, feature engineering including K-Means clustering, and model training with hyperparameter tuning. The retrained model would be evaluated against the current model on a held-out test set before replacement.

6.2.4 Version Control and Governance

Every model version would be tracked. Each serialised model file would be named with a version number and date (e.g., `rf_model_v2_2026-10-01.joblib`) and stored alongside a changelog documenting what changed, why, and the evaluation metrics for that version. The source code, training scripts, and evaluation results would remain in the project git repository, ensuring full reproducibility of every deployed model.

6.2.5 Maintenance Responsibilities

For a small-scale academic project, a single data scientist or analyst would be sufficient to handle monitoring and maintenance. The responsibilities would include:

- Reviewing the weekly performance dashboard and investigating any alerts.
- Running the monthly drift detection scripts and documenting results.
- Executing the quarterly retraining cycle and validating the new model.
- Updating the documentation when the model or pipeline changes.

If the project were to expand to a larger deployment, these responsibilities would be distributed across a team with dedicated roles for data engineering, model validation, and stakeholder communication.